HEIST-IT!

Bop-It! Design Project Final Report

Group Phi: Andrew White | Omar Morsy | Sara Keriakes

Table of Contents

1. Design Overview
- 1.1 High-Level Description
- 1.2 Original Design Concepts
- 1.3 Final Design Summary

2. Design Verification
- 2.1 Test Plan Overview
- 2.2 Subsystem Verification
  - 2.2.1 Rotary Encoder - Dial It!
  - 2.2.2 Reed Switch - Fire It!
  - 2.2.3 VCNL4040 Proximity Sensor - Jiggle It!
  - 2.2.4 OLED Display
  - 2.2.5 Servo Motor
  - 2.2.6 DFPlayer PRO Audio
- 2.3 Integration Testing

3. Electronic Design Implementation
- 3.1 PCB Schematic Description
  - 3.1.1 Microcontroller and Support Circuitry
  - 3.1.2 Sensor Connections
  - 3.1.3 Output Devices
- 3.2 PCB Layout Description
- 3.3 PCB Layout Challenges

4. Software Implementation
- 4.1 Code Structure Overview
- 4.2 Subroutine Listing
- 4.3 Notable Software Design Decisions
  - 4.3.1 Arm Mechanism for Sensor Debouncing
  - 4.3.2 Wrong Action Detection Order
  - 4.3.3 VCNL4040 Separation
  - 4.3.4 Timer Progression

5. Enclosure Design
- 5.1 Design Concept
- 5.2 Structure and Makeup
- 5.3 Manufacturing Process

6. Assembly and System Integration
- 6.1 Assembly Steps
- 6.2 Challenges Encountered

7. Design Testing
- 7.1 Test Plan

7.2 Test Results
7.3 Debugging Process
8. Budget and Cost Analysis
8.1 Bill of Materials
8.2 Cost Per Unit (Prototype)
8.3 Production Cost Estimate (10,000 Units)
9. Team
9.1 Team Members and Roles
9.2 Team Communication and Process
10. Project Timeline
11. Summary, Conclusions and Future Work
11.1 Summary
11.2 Conclusions
11.3 Future Work
References
Appendix I

1. Design Overview

1.1 High-Level Description

Heist-It! is a fast-paced game inspired by the Bop-It! toy, reimagined as a safe cracking heist experience. The player takes on the role of a thief attempting to break into a safe to steal a diamond. The game presents three randomized commands: Dial It!, Fire It!, and Jiggle It! and the player must perform the correct physical action before the timer expires. As the player scores points, the time allowed per command decreases, increasing the difficulty. The game ends either when the player reaches a score of 99 (triggering the safe to unlock and the diamond to be revealed) or when the player runs out of time or performs the wrong action (triggering a game over sequence).

The system is built around an ATmega328P microcontroller and integrates multiple sensors, actuators, audio output, and a display, all housed in a custom-built enclosure designed to look like a miniature bank safe.

1.2 Original Design Concepts

We initially considered several approaches to the three game commands:

Dial It!: We considered using a rotary encoder, a potentiometer, or an accelerometer to detect dial rotation. The B100K rotary potentiometer was selected because it provides a simple analog voltage output that can be read directly by the microcontroller's ADC, requires no additional decoding circuitry, and physically resembles a safe dial.

Fire It!: We considered using an IR break beam sensor, an ultrasonic sensor, or a magnetic reed switch to detect the placement of the dynamite prop. The Littelfuse 59140-1-S-02-A magnetic reed switch was chosen because it provides a clean, noise free digital signal when a magnet approaches, is compact enough to mount discreetly in the enclosure, and has a well-defined activation distance of 6.8mm with the matching actuator magnet.

Jiggle It!: We considered using a physical contact switch, a capacitive touch sensor, or a proximity sensor inside the keyhole. The VCNL4040 proximity sensor was selected because it detects objects without physical contact using infrared reflection, can be mounted inside the keyhole without blocking the key's path, and provides a 16-bit digital reading via I2C which gives fine grained control over detection sensitivity.

Audio: The team evaluated using a simple piezo buzzer, a DFPlayer Mini, or the DFRobot DFPlayer PRO (DFR0768). The DFPlayer PRO was selected because it has 128MB of onboard storage (eliminating the need for an SD card), supports high quality MP3 and WAV playback, and communicates via UART using simple AT commands.

Display: A 0.96-inch SSD1306 OLED display was selected for its small footprint, high contrast, I2C interface (reducing pin usage), and ability to display both large score digits and text commands clearly.

1.3 Final Design Summary

Our final design uses the ATmega328P-PU as the central microcontroller, operating at 16MHz with an external crystal. The three game commands are detected via the rotary encoder (Dial It!), reed switch (Fire It!), and VCNL4040 proximity sensor (Jiggle It!). A servo motor physically locks and unlocks the safe door. Audio feedback is provided by the DFPlayer PRO module. The score and commands are displayed on the OLED. Green and red LEDs provide visual feedback for correct and incorrect actions. The game logic runs entirely on the ATmega328P with no external processor. Fig.0 illustrates the intended design process.

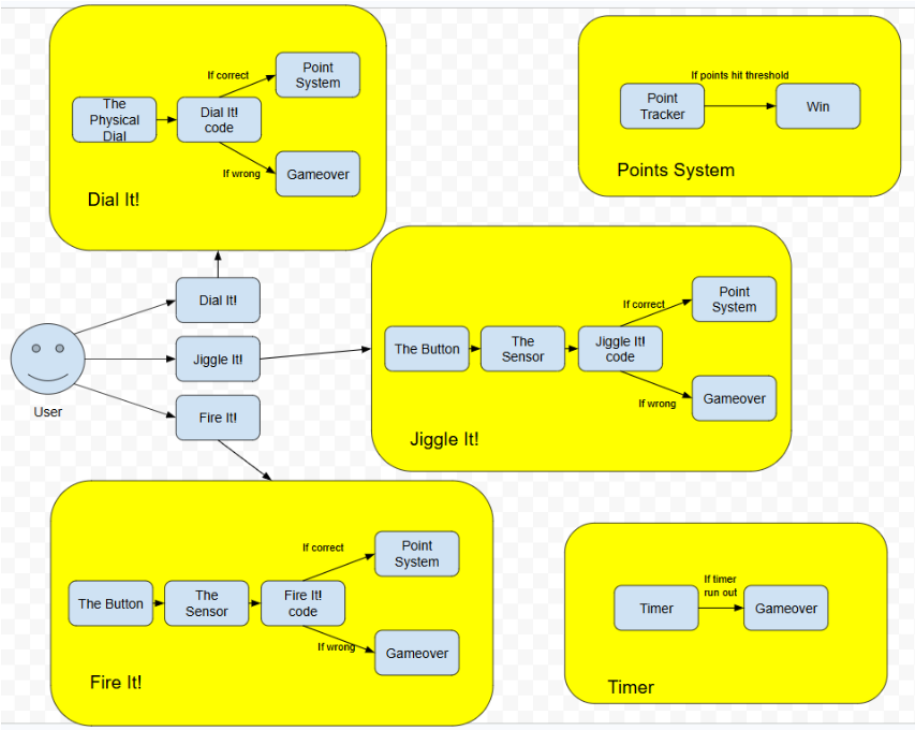

Figure 0 - Basic schematic of system functionality

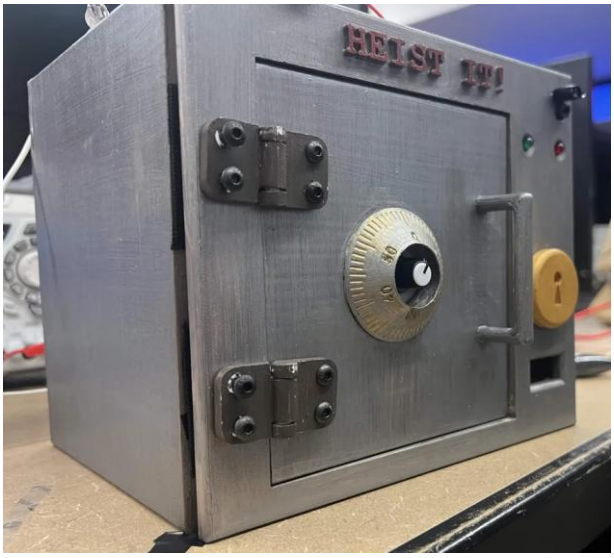

Figure 1 - Final Design

2. Design Verification

2.1 Test Plan Overview

Design verification was conducted in two phases: breadboard prototype testing and final PCB testing. Each subsystem was tested independently before full system integration testing was performed.

Video of breadboard prototype testing:
https://www.youtube.com/shorts/Cqoyjwxn_MA

Video of final PCB testing:
https://www.youtube.com/shorts/6XSmAtiGyGE

2.2 Subsystem Verification

2.2.1 Rotary Encoder Dial It!

The encoder output was connected to pin A0 (PC0) of the ATmega328P. The analog voltage was monitored via Serial Monitor at 9600 baud. The threshold of 512 (mid-scale of the 10-bit ADC) was used to detect rising edges, with each rising edge counted as one click. A minimum of 2 clicks was required to score a point. Testing confirmed reliable detection across multiple rotation speeds.

2.2.2 Reed Switch Fire It!

The Littelfuse 59140-1-S-02-A reed switch was wired with VCC on one side and a 10k pull resistor to pin A1 (PC1) on the other side. When no magnet was present, A1 read LOW. When the magnet approached within the rated 6.8mm activation distance, A1 read HIGH (~1023 on the ADC). Three consecutive analog readings above 700 were required to confirm

a stable trigger and eliminate floating pin noise. Testing confirmed reliable activation and no false triggers in normal operating conditions.

2.2.3 VCNL4040 Proximity Sensor Jiggle It!

The VCNL4040 was initialized via I2C at address 0x60 with maximum IR LED current (200mA) and maximum integration time (8T) for highest sensitivity. The proximity register at address 0x08 was read continuously. A threshold of 2000 was used to distinguish object present from object absent states. A jiggle was counted as one rising edge on the proximity reading. An arm mechanism was implemented to prevent false counts at command start. Testing confirmed detection of a hand or key at approximately 3-5cm from the sensor face.

2.2.4 OLED Display

The SSD1306 OLED was initialized via I2C at address 0x3C using a custom driver without external libraries. A 5x7 pixel font was hand-coded for text rendering and a large 16x24 pixel custom font was coded for the score display. All game screens (waiting, command, score, win, lose) were verified visually on the breadboard prototype. Font bitmaps for digits 3, 5, 6, 8, and 9 were individually verified and corrected during testing.

2.2.5 Servo Motor

The servo was attached to pin D10 and controlled using the Arduino Servo library. Lock position was set to 0 degrees and unlock position to 90 degrees. The servo was verified to reliably reach both positions with a 300ms settle delay. Physical integration with the safe door mechanism was verified during enclosure assembly.

2.2.6 DFPlayer PRO Audio

The DFPlayer PRO (DFR0768) communicates via UART at 115200 baud using AT commands. Six audio tracks were copied to the internal 128MB storage in order. The module was initialized with AT+FUNCTION=1 (music mode), AT+VOL=20 (volume), and AT+PAUSE (stop auto-play). Individual tracks were triggered using AT+PLAYNUM=N commands. Audio playback was verified for all six tracks in the correct game contexts.

2.3 Integration Testing

After all subsystems passed individual testing, the full game loop was verified on the breadboard prototype. The following test procedure was followed:

We power on and verify OLED shows PRESS SW1 TO START GAME with green LED heartbeat. We press start button or turn encoder to start game and verify score resets to 0 and servo locks. For each of the three commands: verify correct audio plays, correct OLED text appears, correct action scores a point, and wrong actions trigger game over. We Verify timer decreases by 50ms after each correct action. We verify game over sequence (red LEDs, CAUGHT! message, servo stays locked). We Verify win sequence (green LEDs, YOU WIN! message, servo unlocks).

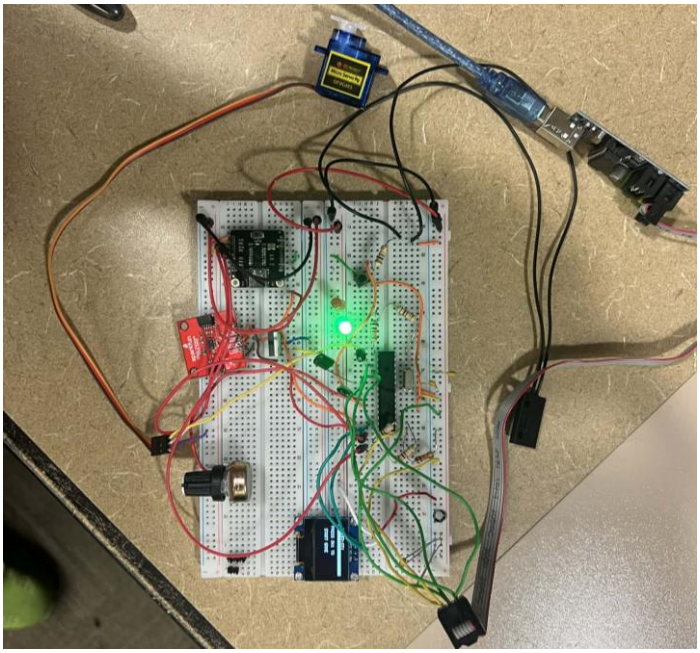

Figure 2 - Breadboard Testing

3. Electronic Design Implementation

3.1 PCB Schematic Description

The schematic was designed around the ATmega328P-PU microcontroller in DIP-28 package. The following key design decisions were made:

3.1.1 Microcontroller and Support Circuitry

A 16MHz crystal (Y1) with two 22pF load capacitors (C1, C2) provides the external clock for maximum operating frequency. Decoupling capacitors C3, C4, C5 (100nF each) are placed on the VCC power rail near the microcontroller to suppress switching noise. C8 and C9 (100nF) decouple the AVCC and AREF pins respectively. R1 (10k) provides a pull-up on the RESET pin to prevent spurious resets. S1 is the main power switch. S2 is the game start button connected to pin PD2 (D2) with INPUT_PULLUP.

3.1.2 Sensor Connections

The rotary encoder (Dial) connects its output to PC0 (A0). VCC and GND rails are provided via the connector. The reed switch (SW3) has one side connected to VCC and R1 (10k) to PC6 (RESET), and the other side connected to R5 (10k) and PC1 (A1). This creates a pull-up circuit where A1 reads HIGH when the switch closes. The VCNL4040 (Jiggle) connects via I2C (SDA to PC4, SCL to PC5) through the Jiggle connector. The 3.3V regulator (U1) with R11 (1k) provides the regulated supply voltage for this sensor.

3.1.3 Output Devices

D1 (green LED) and D2 (red LED) are connected to PB0 (D8) and PB1 (D9) respectively through current-limiting resistors R3 and R4 (1k each). The servo (J1) connects its signal wire to PB2 (D10). The OLED display connects via I2C (shared SDA/SCL bus with

VCNL4040) through the OLED connector. The DFPlayer PRO connects via UART through R6 (1k) on the TX line and R7 (1k) on the RX line to PD0 (D0) and PD1 (D1) respectively. P1 carries the left audio output and P2 carries the right audio output to the speaker.

3.2 PCB Layout Description

The PCB layout was designed using KiCad. The board measures approximately 23mm x 24.5mm and uses a two-layer stackup with red top copper and blue bottom copper traces. The ATmega328P is placed centrally on the board for efficient routing to all peripherals. The crystal Y1 and its load capacitors C1 and C2 are placed immediately adjacent to the XTAL1 and XTAL2 pins to minimize trace length and reduce clock noise. Decoupling capacitors C3, C4, C5 are grouped together near the power entry point on the left side of the board. All sensor connectors (OLED, Jiggle, Dial, Reed Switch SW3) are placed along the bottom edge for easy cable routing inside the enclosure.

The DFPlayer connectors P1 and P2 and resistors R6, R7 are placed in the top right corner near the UART pins of the microcontroller.

The servo connector J1 is placed on the left side near the PWM-capable pin PB2. Power and ground planes were used on the bottom copper layer where possible to reduce impedance and improve noise immunity.

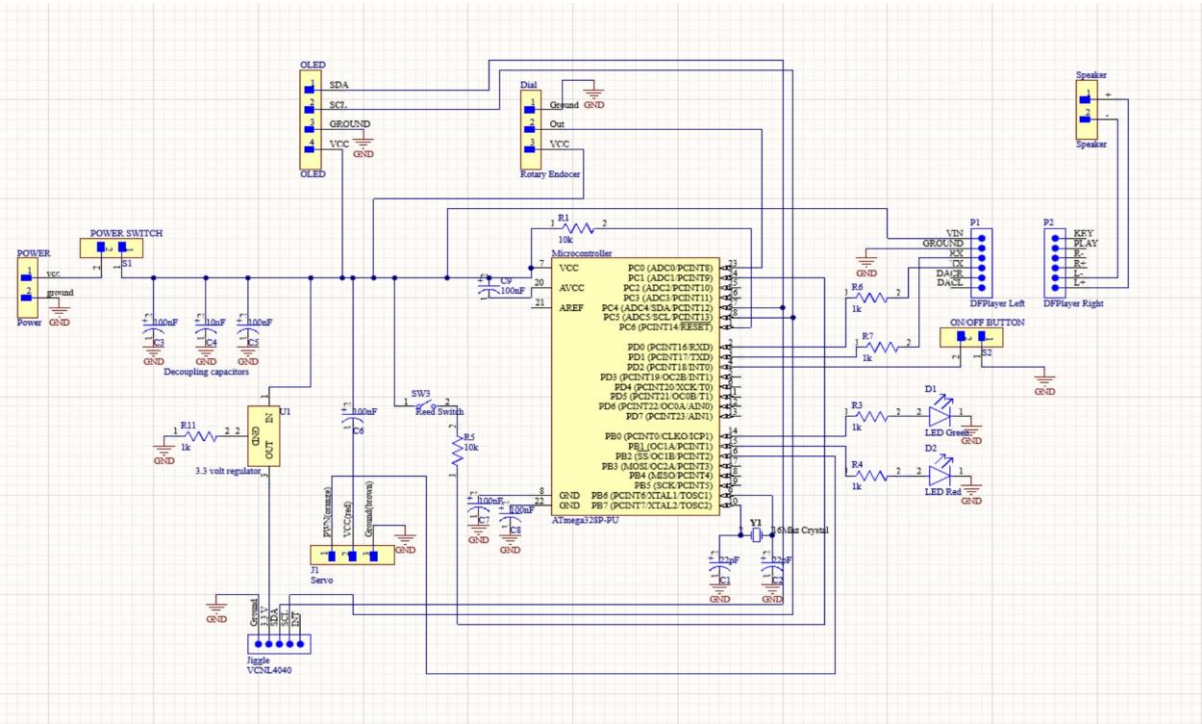

Figure 3 - PCB Schematic

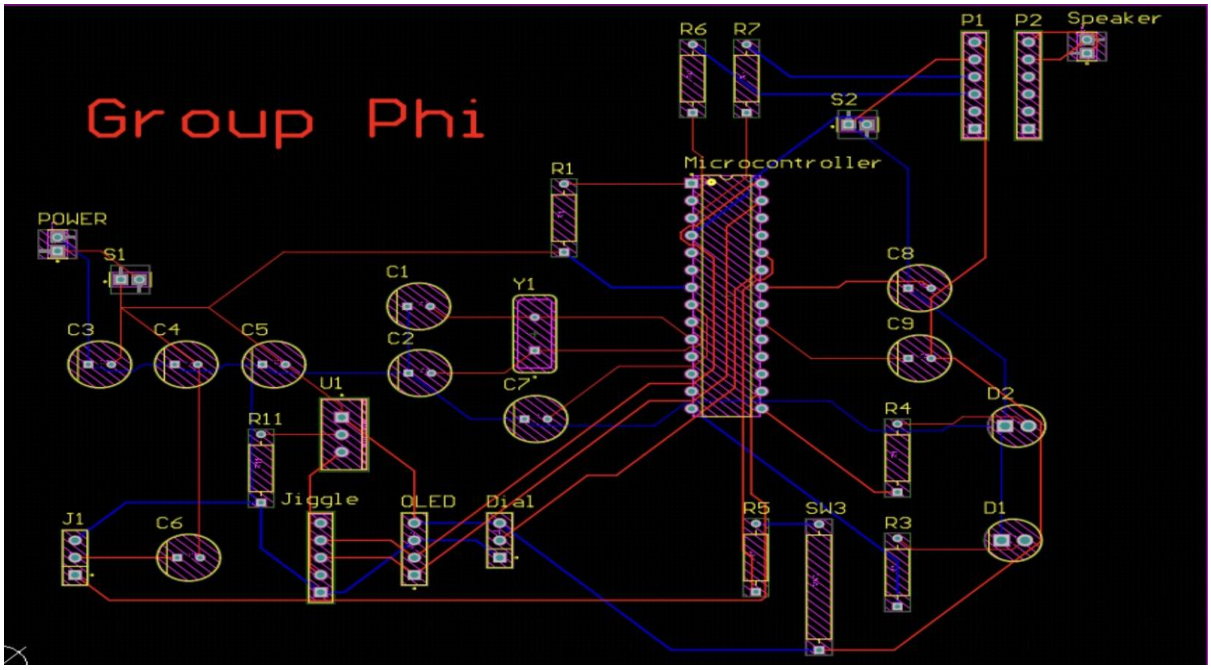

Figure 4 - PCB Design

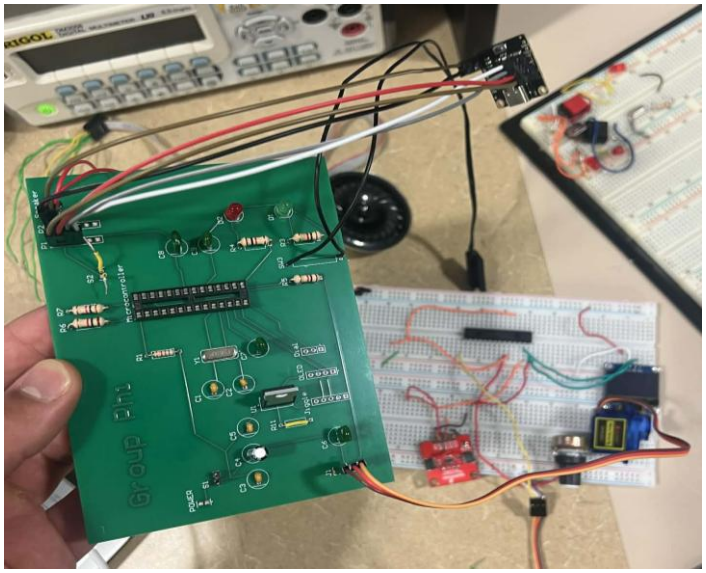

Figure 5 - PCB Soldering Process

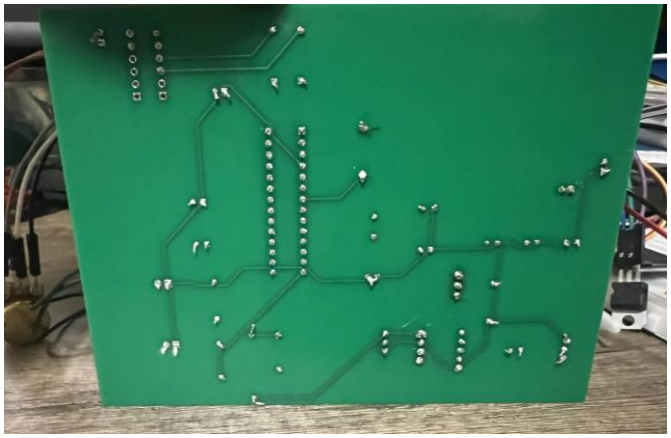

Figure 6 - PCB Post Solder Back

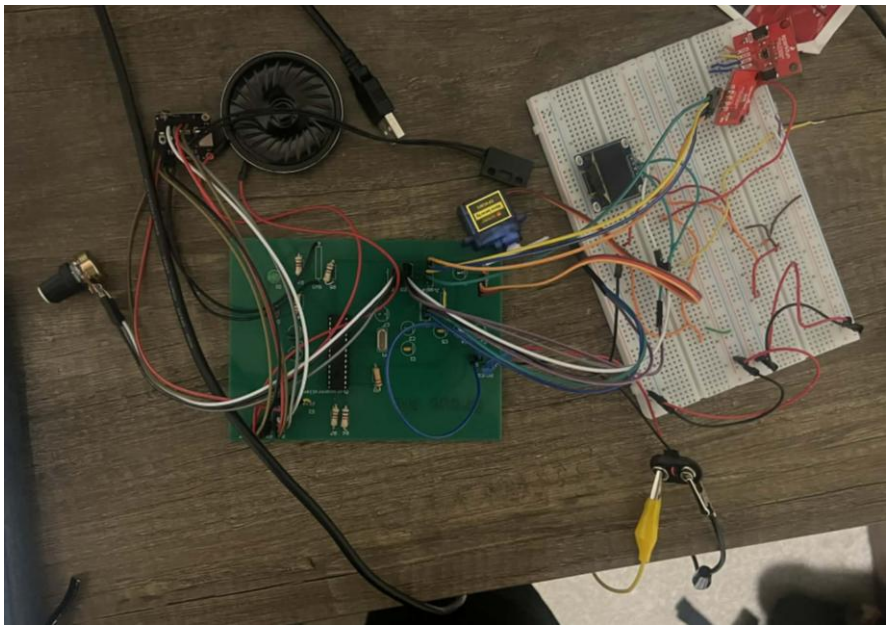

Figure 7 - PCB Post Solder Front

3.3 PCB Layout Challenges

During the creation of the PCB, we had to submit the PCB for fabrication before we could test everything together on the breadboard which led to some issues. We had to remove some parts and add some parts to the PCB when it arrived. We had to remove C6, C7, and R11 because they effected the circuit in a way that made the system not work together. We also needed to add a 5-voltage limiter connected from power to the switch location to make the circuit compatible with a 9-volt battery because a 5-volt battery was not available.

4. Software Implementation

GitHub repository link: https://github.com/omaradelmorsy/Heist-It-

4.1 Code Structure Overview

The firmware is written in C++ for the Arduino/AVR platform and compiled using Arduino IDE 1.8.19 with the ATmega328P board target. The code is structured into the following functional layers:

- Hardware abstraction layer; Direct register access functions for I2C (i2cSend, i2cSend16, i2cGet), OLED driver (oledCmd, oledData), and sensor reading (magnetClose, reedTriggered, pollEncoder).
- Audio layer: AT command functions for the DFPlayer PRO (dfSend, play, setupAudio).
- Display layer; OLED rendering functions (oledInit, oledClear, showScore, showCommand, showMessage, showWaiting) with hand-coded 5x7 and large-digit bitmap fonts.

- Game logic layer; State machine functions (pickMove, giveCommand, gotItRight, gotItWrong, win, lose, startNewGame) and the main loop.
- Sensor arm layer: Debounce and arm logic (pollJiggle, pollFireArm, pollVcnlArm, resetJiggle) that prevents false triggers at the start of each command.

4.2 Subroutine Listing

Function	Description
greenOn() / redOn() / ledsOff()	Control green and red LEDs
blink(green, times, ms)	Blink LED a specified number of times
lock() / unlock()	Move servo to locked (0°) or unlocked (90°) position
resetDial() / getDial()	Reset and read encoder click count since last reset
i2cSend(addr, reg, val)	Write single byte to I2C register
i2cSend16(addr, reg, low, high)	Write 16-bit value to I2C register
i2cGet(addr, reg)	Read 16-bit value from I2C register
dfSend(cmd)	Send AT command string to DFPlayer PRO via Serial
play(track)	Play specific track number on DFPlayer PRO
setupAudio()	Initialize DFPlayer PRO via UART at 115200 baud
setupProximity()	Initialize VCNL4040 proximity sensor registers
magnetClose()	Return true if VCNL4040 reading exceeds threshold
reedTriggered()	Return true if reed switch reads HIGH on 3 consecutive reads
pollEncoder()	Detect encoder rising edges and update dialCount or gameStart
resetJiggle()	Reset all jiggle/fire arm flags and counters
pollJiggle()	Count VCNL4040 proximity rising edges when jiggleArmed
pollFireArm()	Arm fire detection once reed reads clear
pollVcnlArm()	Arm VCNL wrong-action detection once sensor reads clear
oledInit()	Send SSD1306 initialization command sequence
oledClear()	Clear all 8 pages of OLED display memory
oledChar(c)	Render single character using 5x7 font bitmap
oledText(col, page, str)	Render string at specified OLED position
oledBigDigit(col, page, digit)	Render large 16x24 score digit
showScore(s)	Display current score in large digits on OLED
showCommand(cmd)	Display game command text on OLED
showMessage(line1, line2)	Display two-line message centered on OLED
showWaiting()	Display idle waiting screen on OLED

pickMove()	Return random Move enum value (DIAL, FIRE, or JIGGLE)
giveCommand(m)	Reset sensors, play audio, and display command for move m
gotItRight(m)	Return true if correct action detected for move m
gotItWrong(m)	Return true if wrong action detected for move m
win()	Trigger win sequence: audio, unlock servo, display message
lose()	Trigger lose sequence: audio, keep locked, display message
startNewGame()	Reset all game variables and begin new game

Figure 8 - Table of functions used and their definitions

4.3 Notable Software Design Decisions

4.3.1 Arm Mechanism for Sensor Debouncing

A key challenge was preventing false triggers at the moment a command appears. When a command is shown, the sensors may already be in a triggered state due to ambient conditions. To solve this, an arm mechanism was implemented for all three sensors. Each sensor must first confirm a clear (untriggered) state before it can register a player action. This eliminates instant false scores and ensures the player must perform a deliberate action.

4.3.2 Wrong Action Detection Order

Wrong action detection is checked before correct action detection in the main loop. This ensures that if a player performs the wrong action at the exact moment a correct reading could also occur due to sensor noise, the wrong action takes priority and triggers a game over. This prevents any possibility of accidentally scoring from a wrong action.

4.3.3 VCNL4040 Separation

The VCNL4040 proximity sensor is polled only during JIGGLE IT commands for scoring purposes, and only during DIAL IT and FIRE IT commands for wrong action detection. This strict separation prevents the Jiggle sensor from interfering with Fire and Dial scoring, a bug that was present in earlier versions of the code where pollJiggle() ran every round regardless of the current command.

4.3.4 Timer Progression

The game starts with 60 seconds per command; we made it 5 seconds after. Each correct answer reduces the allowed time by 50ms. The minimum time is capped at 400ms (0.4 seconds) to prevent the game from becoming physically impossible. This creates a natural difficulty curve where early rounds feel relaxed but later rounds require near-instant reactions.

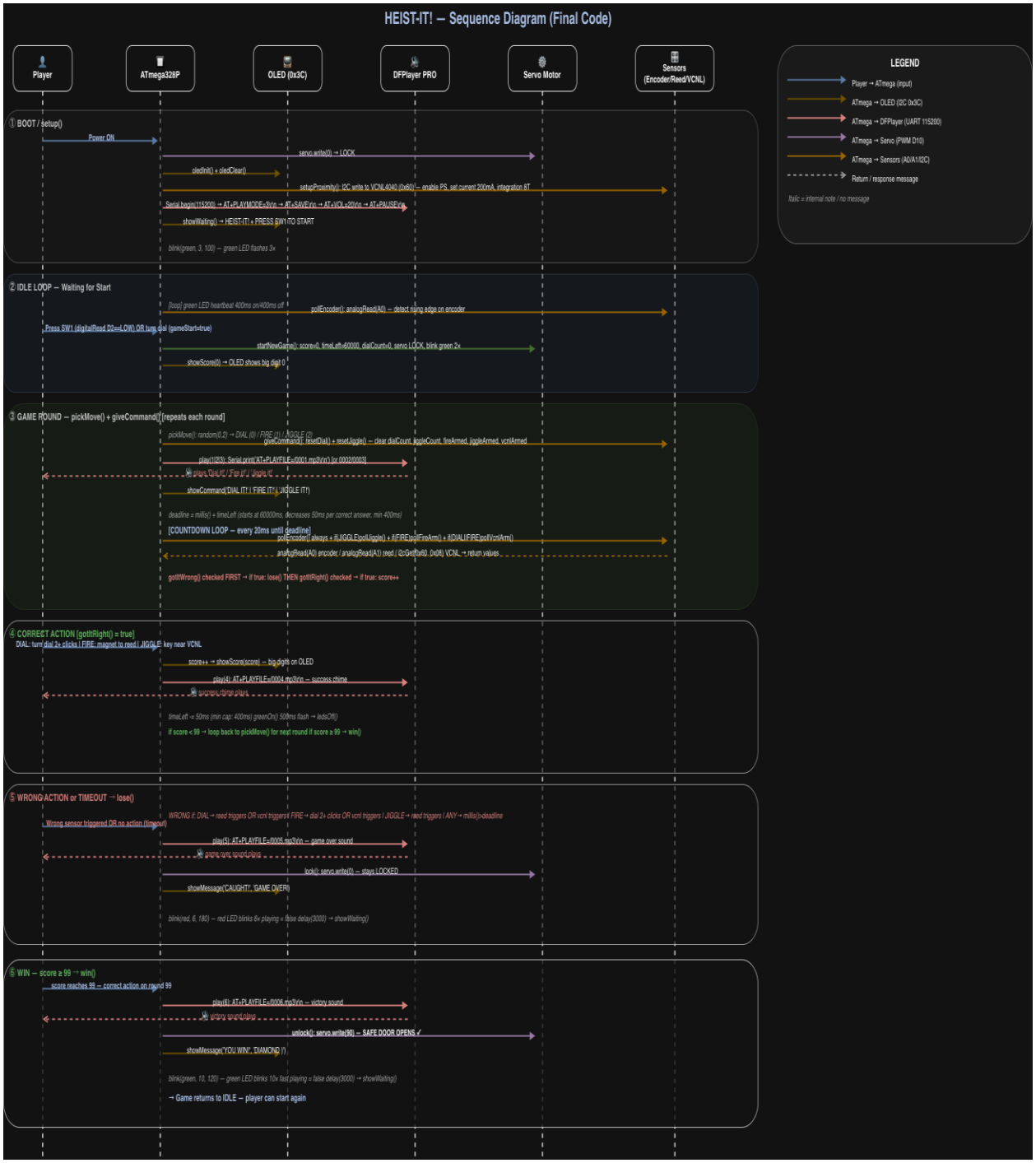

Figure 9 - Sequence Diagrams

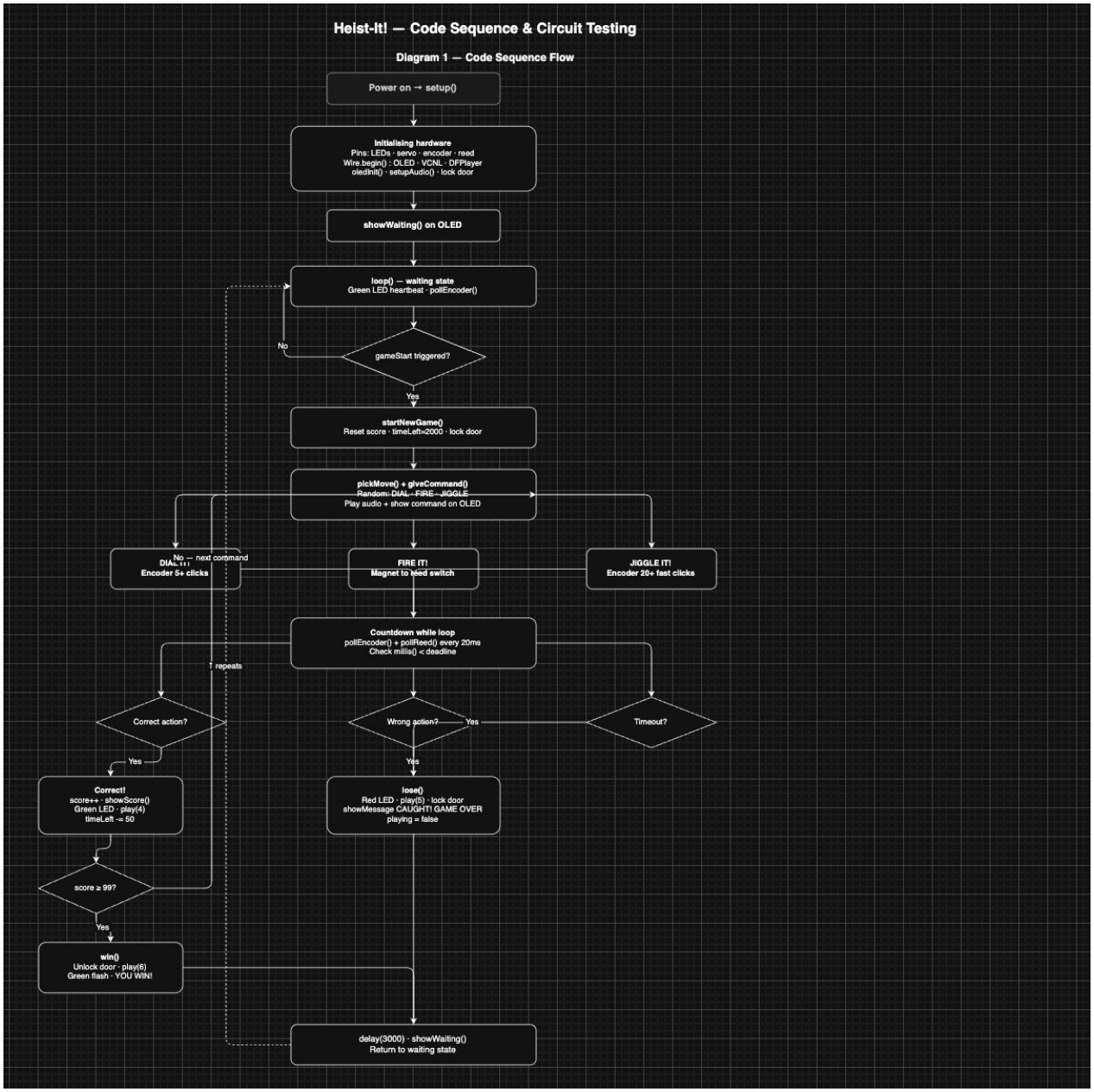

Figure 10 - Code Sequence and Circuit Testing Diagram

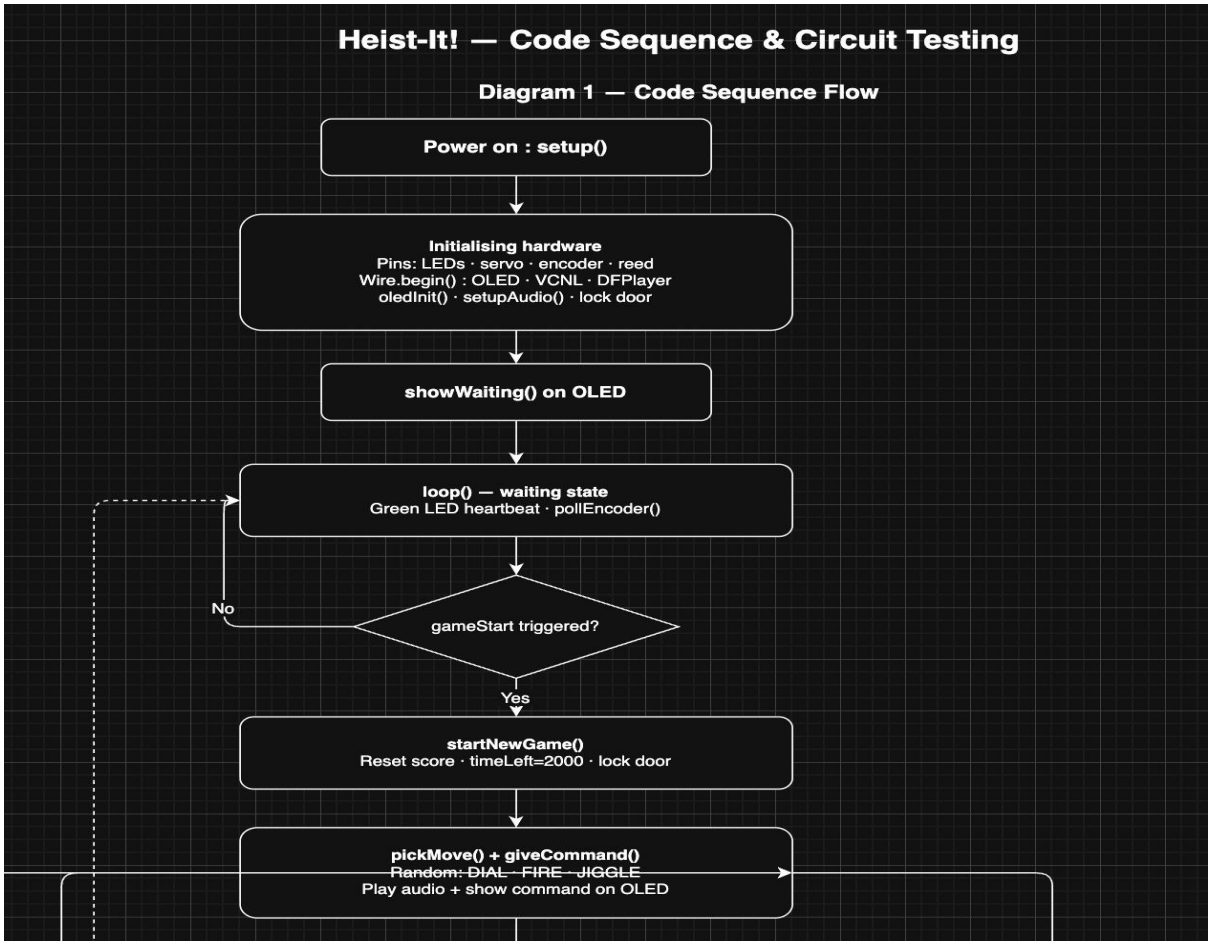

Figure 11a - Code Sequence and Circuit Testing Diagram (Upper Part)

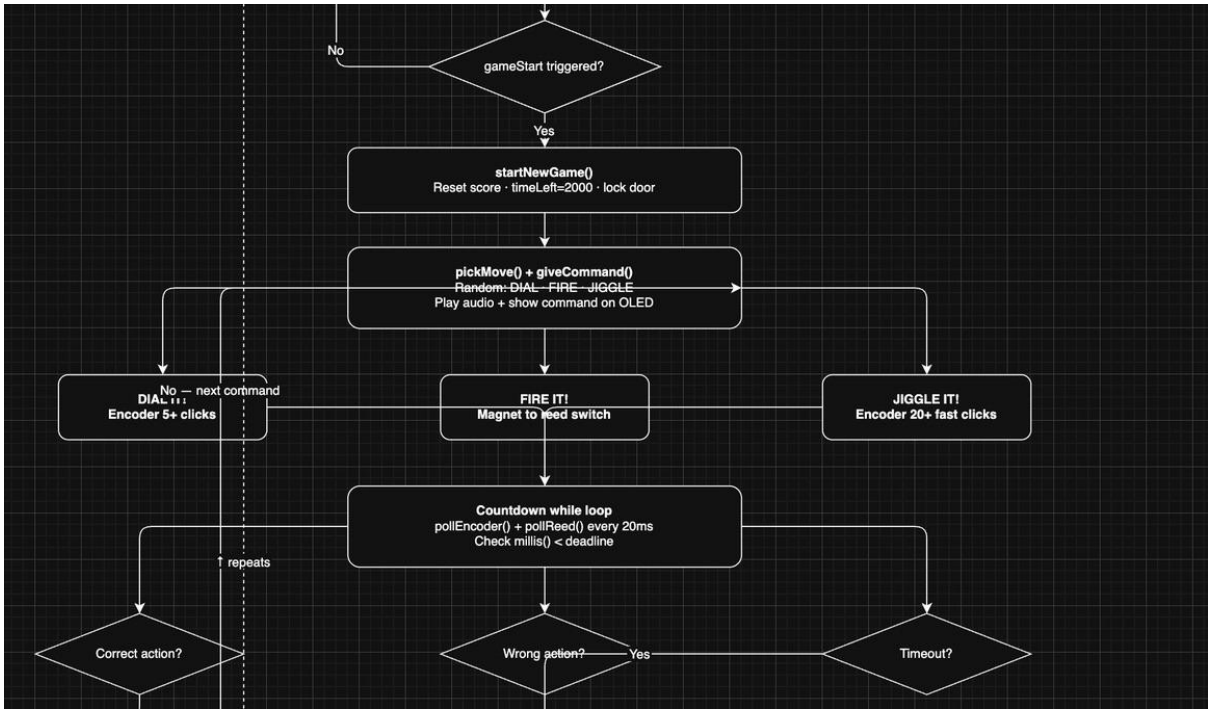

Figure 11b - Code Sequence and Circuit Testing Diagram (Middle Part)

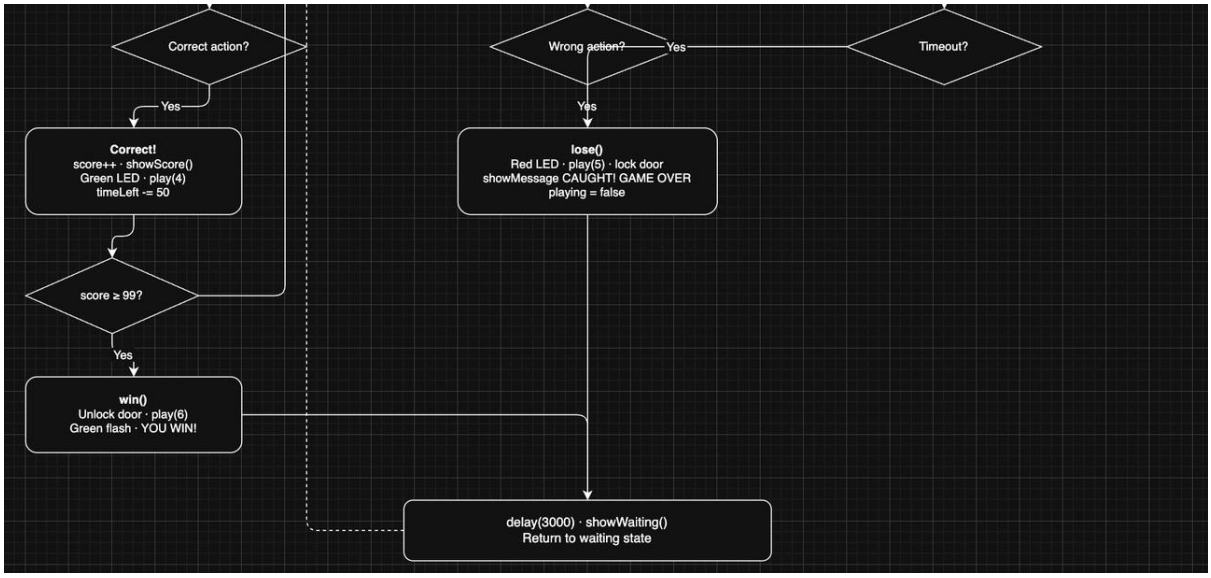

Figure 11c - Code Sequence and Circuit Testing Diagram (Bottom Part)

5. Enclosure Design

5.1 Design Concept

The enclosure was designed to resemble a miniature bank safe, reinforcing the heist theme of the game. The final design features a rectangular metal-look box with a front panel containing the dial, keyhole, OLED display, LEDs, and speaker grille. The top panel has two mounting points for the dynamite sensor (reed switch). The safe door is connected to the servo motor and physically opens when the player wins.

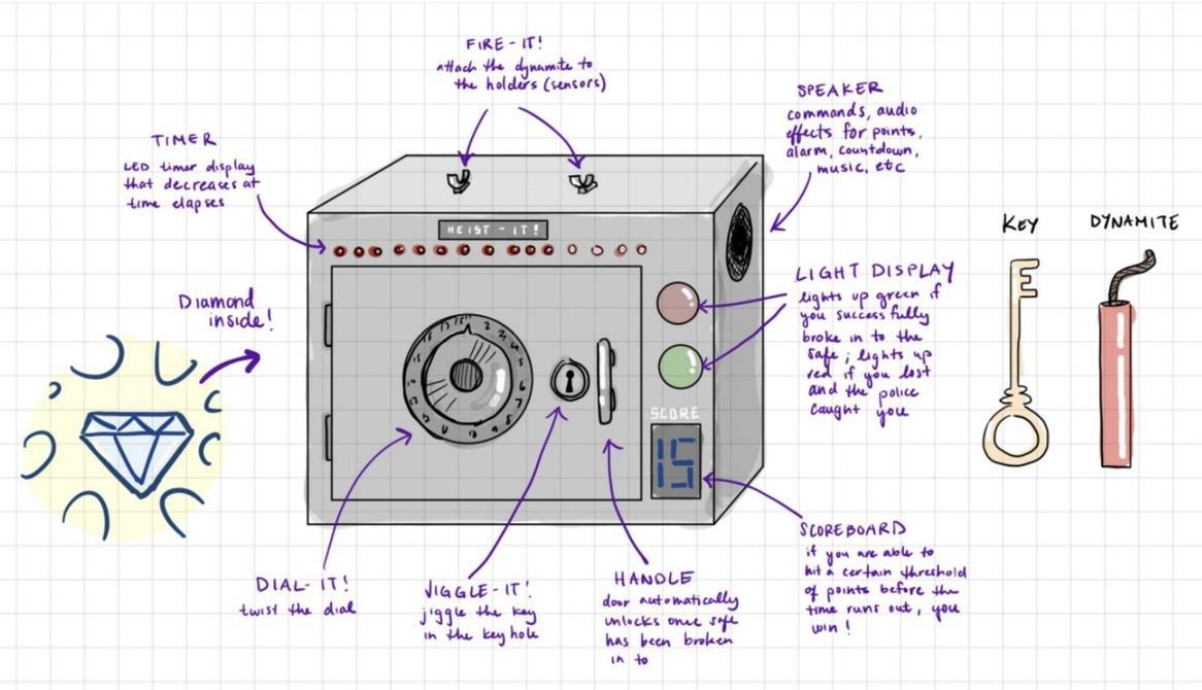

Figure 12 - Enclosure Concept Design

5.2 Structure and Makeup

Due to inexperience with 3D printing and Autodesk Fusion 360, we first made a physical prototype of the design with foam core to understand exact dimensions and constraints of the enclosure. There was much to consider since we wanted to be able to easily access the circuity as well as having it be durable and portable.

We wanted to optimize the placement of all the main circuitry, so we made a large cavity in the right side of the box to house everything. To make the inner box easily removable from the outer box, we added beams to the top to allow the boxes to slide into place snugly, which also allowed for a small cavity on the top of the box for the reed switch to be threaded through.

We measured all components to make sure that they would properly fit in the wall cutouts. The next big thing to think about was how the inner components would fit spatially so that they would not interfere with each other and that they would properly line up with the outer cutouts on the right panel. We placed the components in a way that seemed to accommodate to the user experience, such as placing Power and Start at the very top, the score on the bottom, and the sensors centrally to the user.

The enclosure was constructed with 3D printed PLA for all components and parts besides the outermost box, which was printed with PETG. All walls were built with a 0.25in thickness to increase durability, and because the reference foam core structure walls were 0.25in in thickness as well.

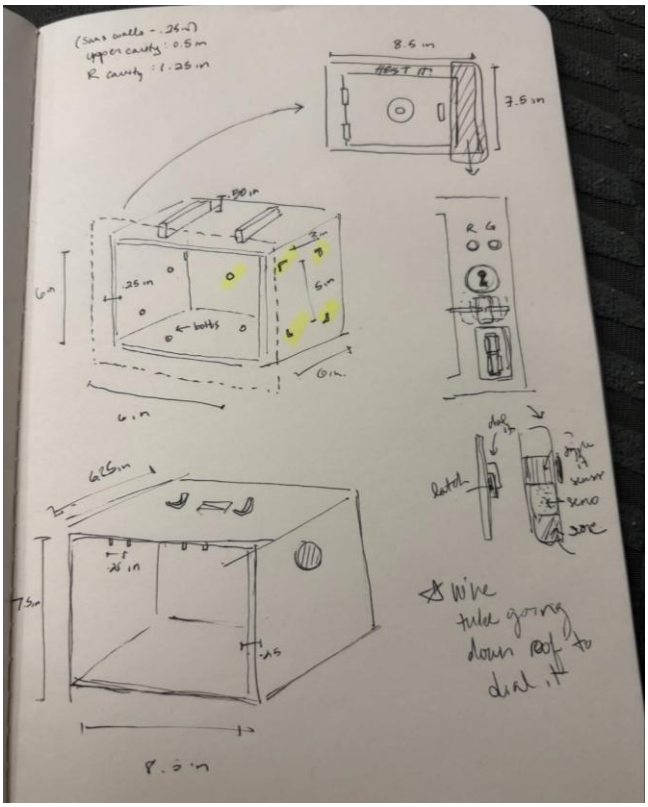

Figure 13 - Enclosure sketch

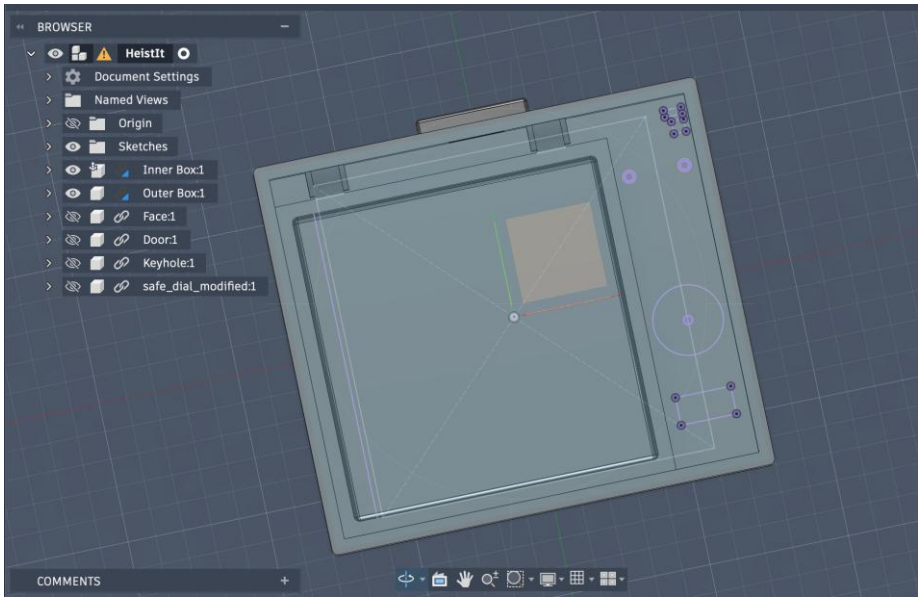

Figure 14 - Inner and outer box, right and upper cavities

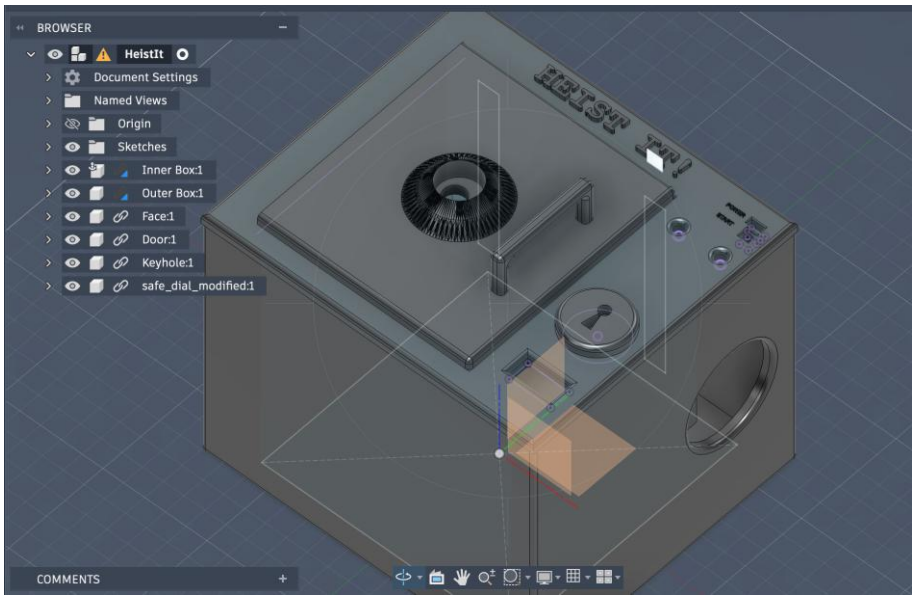

Figure 15 - 3D modelled design in Autodesk Fusion

5.3 Manufacturing Process

The parts were printed at Pitt and CMU Makerspaces/labs. As the parts were extremely snug, the door and front face of the box had to be sanded down. While the original intention was to permanently secure the front face to the inner box, since the inner box beams fit so snugly to the outer box, we were worried about it getting potentially stuck, so we attached the front face with Velcro for ease of access. We 3D printed print-in-place hinges for the door, and found M4 bolts, nuts, and washers to screw them in. We drilled in holes on the front face and door to secure everything. The dial and keyhole were superglued to the front face. For the stick of dynamite, we attached a strand of twine to the 3D printed cylinder, and for the picking tool (as using an actual key would be counterintuitive to a heisting game) we found a small bamboo pick that fit nicely into the keyhole. All parts were painted with acrylic paint

from SERC and bolts were taken from the Makerspace. Components were mainly wired with electrical tape and Velcro, so that they could be easily replaceable and accessed, and wires were managed with electrical tape.

Figure 16 - Assembled Enclosure

6. Assembly and System Integration

6.1 Assembly Steps

The assembly was performed in the following order:

Step 1: PCB fabricated by JLCPCB and received. All through hole components soldered in order: passive components first (resistors, capacitors, crystal), then ICs (ATmega328P, voltage regulator), then connectors.

Step 2: Firmware uploaded to ATmega328P via USBtinyISP programmer before installing in enclosure. Arduino IDE 1.8.19 used with Board: Arduino Uno and Programmer: USBtinyISP.

Step 3: Speaker mounted in enclosure grille and connected to DFPlayer PRO output pins.

Step 4: Sensor components (VCNL4040 breakout, OLED module, reed switch) wired to their respective PCB connectors using ribbon cable.

Step 5: Servo motor mounted inside enclosure and connected to safe door mechanism and PCB connector.

Step 6: Rotary encoder (dial) mounted through front panel and connected to PCB.

Step 7: DFPlayer PRO loaded with audio tracks via USB-C and connected to PCB via UART lines.

Step 8: PCB mounted inside enclosure on standoffs. All cables routed and managed.

Step 9: Full system power-on test performed before closing enclosure.

Step 10: Enclosure closed and final integration test performed.

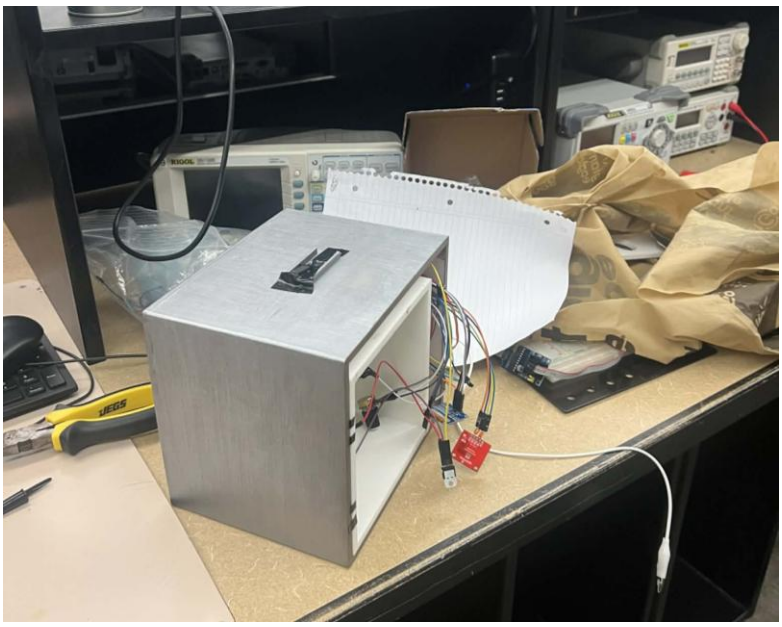

Figure 17 - Assembling Process after step

6.2 Challenges Encountered

There were a lot of issues that came up in this stage.

1. The enclosure was a little too small for certain parts we had because of measurements not taking account for the layers of paint in the design. This decreased the size of already tiny holes we had for parts such as the Power and Start buttons.
2. Because it was extremely difficult to secure a 3D printer due to the sheer amount of people printing out designs for end-of-semester projects, even though the enclosure was completed many days before the deadline, printing was still up until the day prior and there was not enough time to re-print pieces that would've been easily fixed with adjusting dimensions (such as having a cavity for where the bolts were, or accounting for the layers of paint). We had to outsource many components to friends in CMU as all the Pitt printers were very inaccessible.
3. Cable management was a pain in multiple ways. First, we just had the wires be too big for some components and too small for other components which led to issues in not being able to open the door fully. Second, some components were getting stuck and tangled with each other which led to some parts falling out of place during assembling
4. When trying to put the servo in place, there wasn't a spot at all we could place it. Due to last minute adjustments and resulting oversight, the enclosure didn't account for the servo motor. With limited time left to work on the project (there was not enough time

for us to redesign, print, and paint the enclosure), we had to ditch the motor and leave it in the wire mess in the enclosure. This decision hard to make but had to be done.

5. When testing with a normal 9-volt battery, the Dial It! action instantly failed with no input given. We had enough time to fix this but our main programmer who worked on the code didn't have time and had to go somewhere else. This led the rest of us to try to fix the issue but with little success. This led our design to work completely properly on power supplies but not fully with batteries.

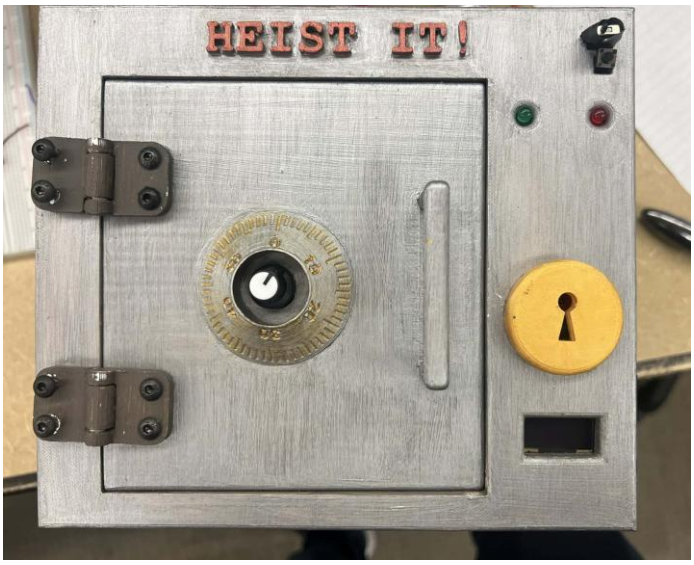

Figure 18 - Final Design Front

Figure 19 - Final Design Inside (Wire Mess)

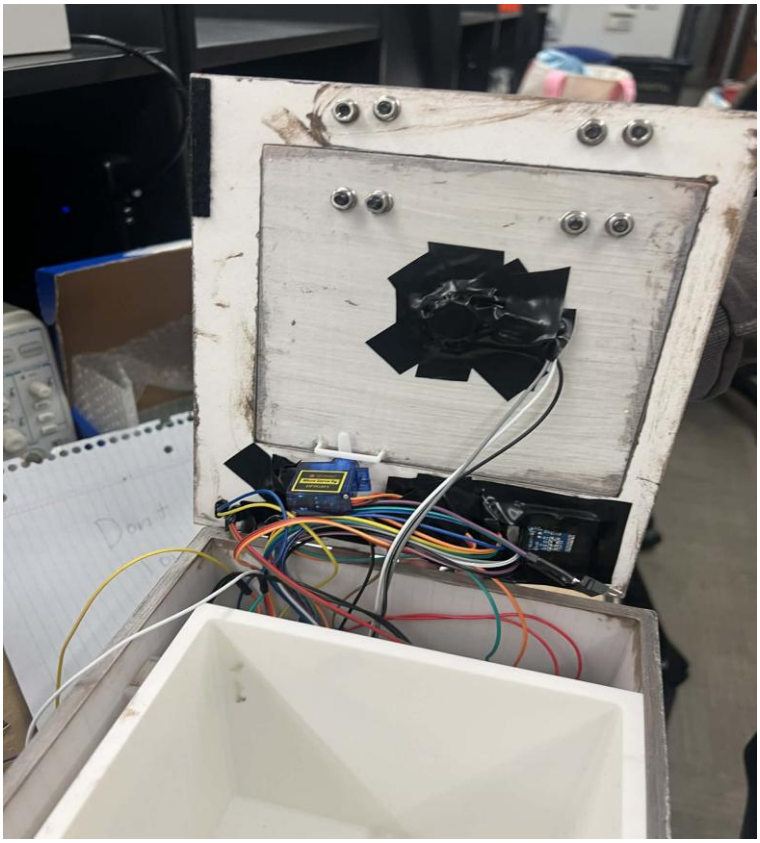

Figure 20 - Final Design Inside (Cover)

7. Design Testing

7.1 Test Plan

Testing was conducted using the 12-step test procedure defined in the design documentation. Each step was performed in order and results recorded. The Serial Monitor at 9600 baud was used during breadboard testing to observe sensor values in real time.

7.2 Test Results

Test	Expected Result	Actual Result	Status
Boot check	OLED shows HEIST-IT, green LED blinks 3x, servo locks	OLED shows HEIST-IT, green LED blinks 3x, servo locks	Working
Start game via encoder	Score resets to 0, OLED shows 0	Score resets to 0, OLED shows 0	Working
Start game via button	Score resets to 0, OLED shows 0	Score resets to 0, OLED shows 0	Working
DIAL IT correct action	2+ encoder clicks : green LED, score +1	2+ encoder clicks : green LED, score +1	Working
DIAL IT wrong (fire)	Reed triggers : GAME OVER	Reed triggers : GAME OVER	Working

DIAL IT wrong (jiggle)	VCNL triggers : GAME OVER	VCNL triggers : GAME OVER	Working
FIRE IT correct action	Magnet close to reed : green LED, score +1	Magnet close to reed : green LED, score +1	Working
FIRE IT wrong (dial)	Encoder 2+ clicks : GAME OVER	Encoder 2+ clicks : GAME OVER	Working
JIGGLE IT correct action	Key near VCNL : green LED, score +1	Key near VCNL : green LED, score +1	Working
JIGGLE IT wrong (fire)	Reed triggers : GAME OVER	Reed triggers : GAME OVER	Working
Timeout	No action : GAME OVER after timer expires	No action : GAME OVER after timer expires	Working
Win condition	Score 99 : servo unlocks, YOU WIN! displayed	Score set to 15 because 99 is too high for a demo. servo unlocks, YOU WIN! displayed	Working

Figure 21 - Table of test results for the final day of testing

7.3 Debugging Process

Several bugs were encountered and resolved during development:

- Bug 1: Start button not working: The start button (S2) was wired in the schematic but digitalRead() was never called in the firmware. Fixed by adding pinMode(START_BTN_PIN, INPUT_PULLUP) and checking digitalRead() in the idle loop.
- Bug 2: FIRE IT auto-scoring: The VCNL4040 proximity sensor threshold of 100 was too low, causing ambient IR reflections inside the enclosure to trigger FIRE IT instantly. Fixed by raising threshold to 2000 and implementing the fireArmed mechanism.
- Bug 3: JIGGLE IT not working: gotItRight(JIGGLE) was checking magnetClose() instead of jiggleCount, completely ignoring the jiggle counter. Fixed by changing the check to jiggleCount >= 1.
- Bug 4: JIGGLE IT scoring during FIRE IT: pollJiggle() was called every round regardless of the current command. Fixed by wrapping it in if (current == JIGGLE) pollJiggle().
- Bug 5: Reed switch floating pin: A1 read random values when the reed switch was open because there was no pull-down resistor. Fixed by reading A1 three times and requiring all readings > 700, and recommending addition of a 100k pull-down resistor to the circuit.
- Bug 6: DFPlayer auto-playing all tracks at boot: AT+FUNCTION=1 caused the module to auto-play all files sequentially. Fixed by adding AT+PAUSE after initialization.
- Bug 7: Wrong action triggering during correct action: gotItRight() was checked before gotItWrong(). Fixed by swapping the order so wrong actions always take priority.

JD Heist-It Demo: Win https://youtube.com/shorts/APzcbXDdsXY?feature=share

JD Heist-It Demo: Lose https://youtube.com/shorts/uB-h6X9AMn4?feature=share

8. Budget and Cost Analysis

8.1 Bill of Materials

Component	Part Number	Qty	Unit Cost	Total
ATmega328P-PU	ATmega328P-PU	1	$2.30	$2.30
16MHz Crystal	ABLS-16.000MHZ	1	$0.45	$0.45
VCNL4040 Breakout	SparkFun SEN-15177	1	$7.50	$7.50
Littelfuse Reed Switch	59140-1-S-02-A	1	$4.40	$4.40
SSD1306 OLED 0.96"	Generic SSD1306	1	$3.50	$3.50
DFPlayer PRO	DFR0768	1	$8.90	$8.90
Micro Servo Motor	SG90	1	$2.50	$2.50
3.3V Voltage Regulator	LD1117-3.3	1	$0.35	$0.35
Resistors (assorted)	Various	11	$0.05	$0.55
Capacitors (assorted)	Various	9	$0.08	$0.72
LEDs (green + red)	Various	2	$0.15	$0.30
Push buttons	Various	2	$0.25	$0.50
Power switch	Various	1	$0.50	$0.50
Speaker	Various	1	$1.50	$1.50
PCB (JLCPCB, 5 units)	Custom	1	$5.00	$5.00
Enclosure materials	Various	a lot	Various	$10.00
Miscellaneous (wire, connectors)	Various	1	$3.00	$3.00

Figure 22 - Table with Bill of Materials

8.2 Cost Per Unit (Prototype)

Total prototype cost: approximately $51.57 per unit (excluding enclosure and assembly labor). Enclosure cost is a placeholder pending manufacturing details.

8.3 Production Cost Estimate (10,000 Units)

At production volume of 10,000 units, significant cost reductions are expected through bulk component pricing and automated PCB assembly (SMT reflow). The following estimates are based on JLCPCB pricing for assembled PCBs and bulk component pricing from major distributors:

Cost Category	Prototype (1 unit)	Production (10,000 units est.)
PCB fabrication	$5.10	$51,000.00
PCB assembly (JLCPCB PCBA)	N/A (hand soldered)	~$12,000 (machine soldered)
Components (BOM)	$36.47	$320,000.00
Enclosure	$10.00	$100,000.00
Audio files / firmware	$0.00 (one-time)	$0.00
Total per unit	~$51.57	~$485,000

Figure 23 - Table of production cost estimates

9. Team

9.1 Team Members and Roles

Team Member	Role	Responsibilities
Andrew White	Hardware and Testing	Making the PCB, found the parts to use, tested the parts on breadboard, testing/debugging, and soldered
Omar Morsy	Software and Testing	Tested the code on the breadboard and testing/debugging
Sara Keriakes	Enclosure and Testing	

Figure 24 -Table of team member roles

9.2 Team Communication and Process

We did almost all of our communication though discord using a server. We scheduled meeting using when2meet which were posted weekly in the meeting-schedule chat. We put links to every document used in the creation of the project in the assignment-document chat. We tried to meet 2 times a week with at least one three-person meeting a week to discuss our progress and what we were working on. As we got closer to the deadline, we met more often (~5-6 times a week).

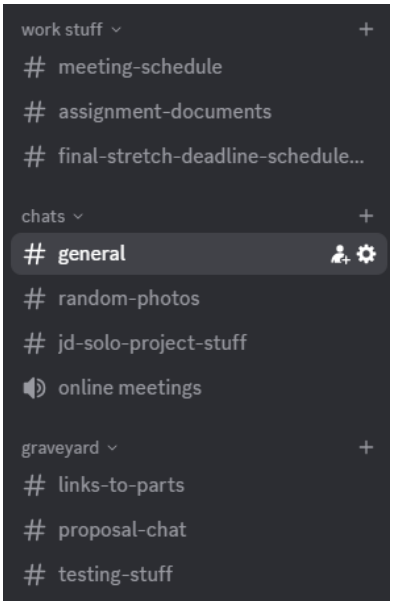

Figure 25 - Discord channels used for communication

10. Project Timeline

Milestone	Target Date	Actual Date	Notes
Project kickoff / Initial design concept	Mar 5th	Mar 5th	The proposal and ideas were made here
Breadboard prototype assembled	Mar 17th	Apr 3rd	Not having all the parts yet delayed it a bit
Schematic completion	Mar 24th	Mar 28th	Not having all the parts yet delayed it a bit
Firmware first working version	Mar 25th	Apr 9th	Lots of issues in the code caused this
PCB layout completion	Mar 29th	Mar 29th	Had to add extra holds just in case testing went wrong
PCB ordered from JLCPCB	Mar 31st	Apr 1st	Gerber file issue leading it to be late
PCB received and assembled	Apr 9th	Apr 11th	The delayed Gerber files caused this
Enclosure design completed	Apr 17th	Apr 18th	Review stage took an extra day
Enclosure fabricated	Apr 19th	Apr 21st	The 3d printers were all being used and it was delayed until we could use them
Full system integration and testing	Apr 21st	Apr 22nd	Waited for enclosure to be printed
Final report submission	Apr 24th	Apr 24th	
Final presentation	Apr 23rd	Apr 23rd	

Figure 26 - Project Timeline

11. Summary, Conclusions and Future Work

11.1 Summary

Heist-It! is a fully functional physical game that successfully implements the Bop-It! game concept in a safe-cracking heist theme. The system integrates three distinct input sensors (rotary encoder, magnetic reed switch, and VCNL4040 proximity sensor), a servo-actuated safe door, an OLED display, audio feedback via the DFPlayer PRO, and visual LED indicators, all controlled by an ATmega328P microcontroller.

The game logic was implemented entirely in C++ without relying on high-level libraries for the display, sensor, or audio drivers, demonstrating a thorough understanding of I2C communication protocols, ADC usage, UART AT command protocols, and embedded systems programming techniques. Multiple hardware and software bugs were identified and resolved during the development process, resulting in a reliable and responsive game experience.

11.2 Conclusions

As a team, we learned that a project of this scale can be challenging, as not everything works as intended. Small setbacks can accumulate and lead to issues later in the process. The most important takeaway was learning how to address these setbacks and collaborate effectively to resolve them in a timely manner. Our teamwork was a strong point throughout the project, and without it, the outcome could have been significantly worse. We communicated well and were able to work together productively. Many of the challenges stemmed from our relative inexperience in our respective areas (software, hardware, and 3D printing). As Computer Engineering students, we found the more Electrical Engineering-focused aspects, such as breadboarding and soldering, particularly difficult. We also learned that final integration is extremely demanding and ultimately proved to be the most challenging phase of the project. Our design goals were to create a game with three actions and a moderate level of polish. While we may have fallen short in terms of polish, we were overall successful in meeting our primary objectives. Toward the end of the project, unforeseen circumstances (such as team members being unavailable due to sickness or military trainings) affected our final outcome and contributed to a result that was not as strong as we had hoped.

11.3 Future Work

If another design iteration were to be performed, the team would consider the following improvements:

- Hardware pull-down resistor on A1: Adding a 100k pull-down resistor between A1 and GND in the PCB schematic to eliminate the floating pin issue on the reed switch, removing the need for the software three-read debounce workaround. This will increase cost but increase efficiency

- VCNL4040 threshold calibration: Implementing an automatic calibration routine at boot that reads the ambient proximity value and sets the threshold dynamically, rather than using a hardcoded value of 2000.
- Wrong action difficulty modes: Adding multiple difficulty levels where the wrong-action penalty can be disabled for beginners or the time reduction per correct answer can be increased for expert mode.
- Score persistence: Adding EEPROM storage to save the high score across power cycles.
- DFPlayer prompt tone: Disabling the DFPlayer PRO startup prompt tone using AT+PROMPT=OFF to prevent an unwanted audio beep at boot.
- Enclosure refinement: Improving the physical integration of the servo and safe door mechanism for smoother and more reliable door opening.
- PCB revision: Moving the reed switch connection away from the I2C bus lines to avoid potential crosstalk, which was the original reason the analog reed pin was problematic.

References

Vishay Semiconductors. VCNL4040 Datasheet, Rev. 2.0, January 2026. https://www.vishay.com/docs/84274/vcnl4040.pdf

Littelfuse. 59140 Series Reed Sensors Datasheet, September 2023. https://www.littelfuse.com

DFRobot. DFPlayer PRO (DFR0768) Wiki. https://wiki.dfrobot.com/DFPlayer_PRO_SKU_DFR0768

Atmel. ATmega328P Datasheet. https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

Solomon Systech. SSD1306 OLED Controller Datasheet. https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf

Appendix I

GitHub repository link: **https://github.com/omaradelmorsy/Heist-It-**